

COSC264 · INTRODUCTION TO COMPUTER NETWORKS AND THE INTERNET · TERM 4

Module 1

Introduction to Routing

1 · The Routing Problem

2 · Shortest Paths & Distance-Vector

3 · Dijkstra & Link-State Routing

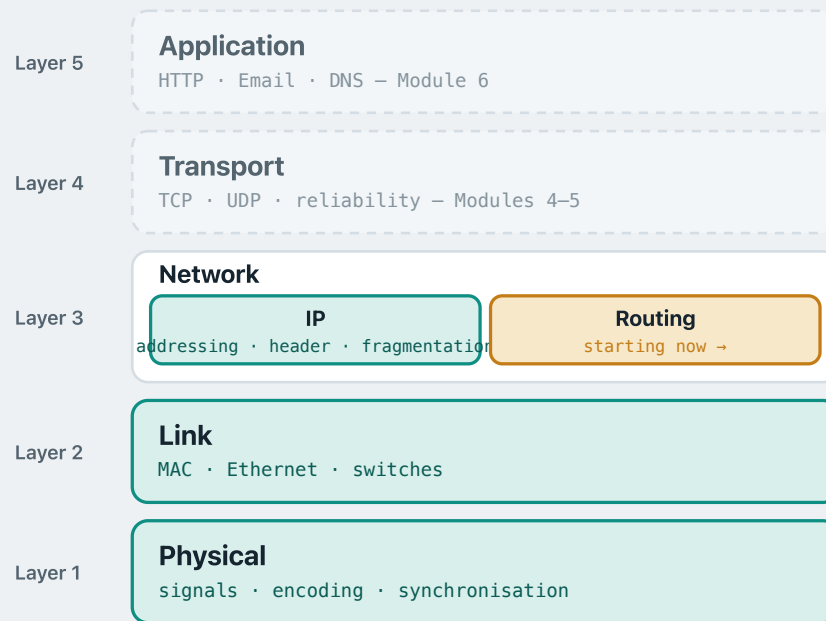
University of Canterbury · Yinuo Zhang · yinuo.zhang@canterbury.ac.nz

Module 1 – Introduction to Routing

cosc264.up.railway.app

21 Sept 2026 · 1 / 43

An overview of the rest of the course



THE FIRST HALF OF THE COURSE

- The Internet stack is divided into **five layers**.
- We covered layers **1** and **2** — physical and link — and half of layer **3**.

FROM HERE ON

- The missing half of layer **3** is **routing** — that is what we start on now.
- The two layers above — **transport** and **application** — follow in the rest of the course.

MODULE 1

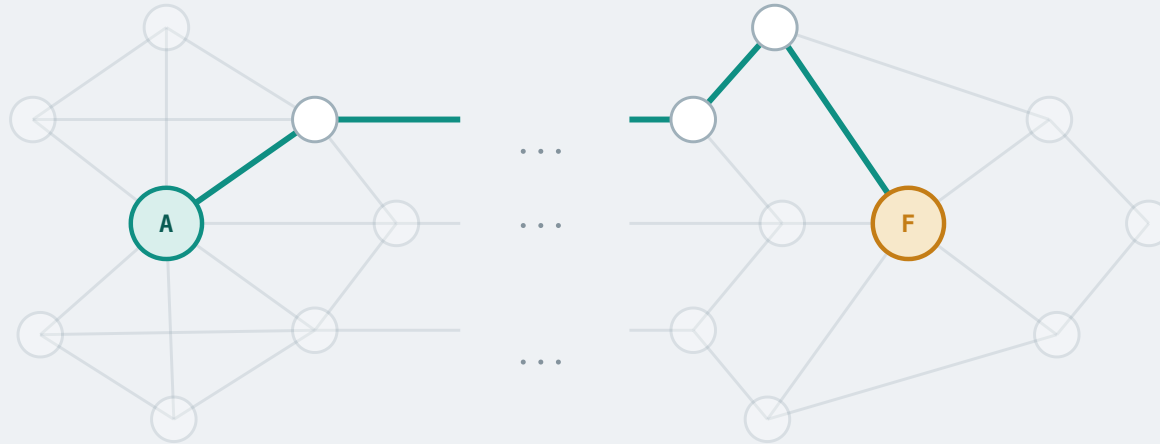
The Routing Problem

1 • The Routing Problem

2 • Shortest Paths & Distance-Vector

3 • Dijkstra & Link-State Routing

Why do we need routing?



DEFINITION A **path** from A to F is a sequence of routers $R_0, R_1, \dots, R_k$ with $R_0 = A$ and $R_k = F$, where every consecutive pair is joined by a link. The message is handed along the whole sequence.

What does routing do?

Routing is how a network decides, **automatically**, which path packets take from each source to each destination.

which path counts as best

Routing does not hand out just *any* path. It defines a **cost metric**, so that every path has a cost — and then picks the path with the **least** cost.

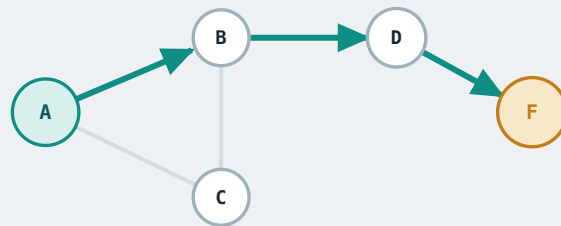
The cost metric can capture many factors, such as hop count (distance), delay, bandwidth, and so on.

when it happens

- **Before any packet arrives.** Working a path out per message, on the fly, would be far too slow — the answer has to be ready and waiting by the time a message turns up.
- **And again whenever the network changes.** Links fail and costs change, so a path that was best a moment ago may not even exist now. Routing is therefore always running.

How is the path information stored? — the forwarding table

Notice that each router holds only a **local** decision — none of them has the global path. Put those local decisions together and you get **A → B → D → F**.



A → B → D → F

A's table

DEST	NEXT HOP
B	B
C	C
D	B
F	B

B's table

DEST	NEXT HOP
A	A
C	C
D	D
F	D

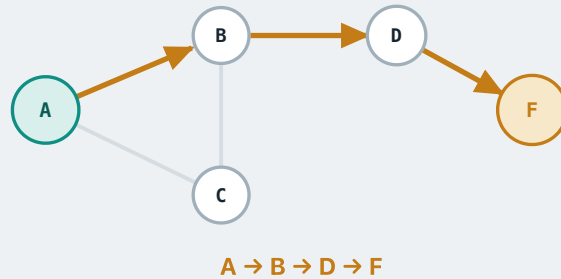
D's table

DEST	NEXT HOP
A	B
B	B
C	B
F	F

The goal of **routing**, therefore, is to build those **forwarding tables**.

Reading one row and passing the message on — what we just did — is **forwarding**.

Routing vs. forwarding



A's table

DEST	NEXT HOP
B	B
C	C
D	B
F	B

B's table

DEST	NEXT HOP
A	A
C	C
D	D
F	D

D's table

DEST	NEXT HOP
A	B
B	B
C	B
F	F

	ROUTING	FORWARDING
job	produces the tables	uses the tables
plane	control plane	data plane
when	before data arrives	when a datagram arrives
speed	slower	faster – nanoseconds

How do we do routing?

Easier to state than to do. Every router knows only the links it is plugged into — **none of them can see the whole network** — and yet a path has to be found across all of it.

a routing algorithm

Solves the routing problem under **ideal assumptions**.

a routing protocol

Embeds a routing algorithm into **a real networking context**:

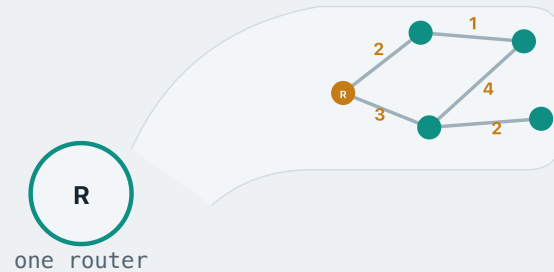
- it operates in a **distributed** environment;
- it carries that **information exchange** between the routers using **existing Internet protocols**;
- that exchange **takes time and can fail**, and the protocol has to allow for it.

Two families of routing algorithms

link-state

distance-vector

Link-state: get the whole map

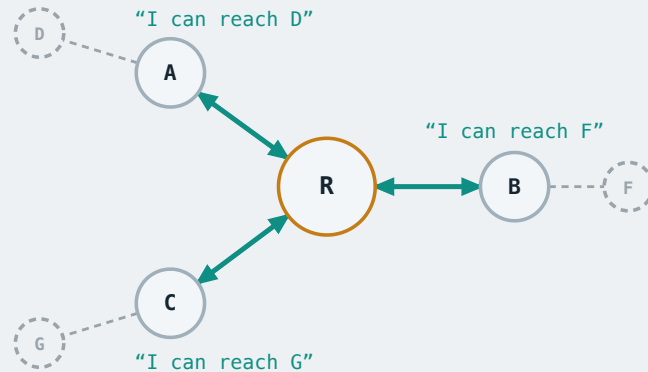


The routers talk to each of their **neighbours**.

They exchange information, so that at the end every router holds a **global view** of the network.

With that map in hand, each router works out its own forwarding table **locally** (next lecture).

Distance-vector: ask your neighbours



R's forwarding table

DEST	NEXT HOP
D	A
F	B
G	C

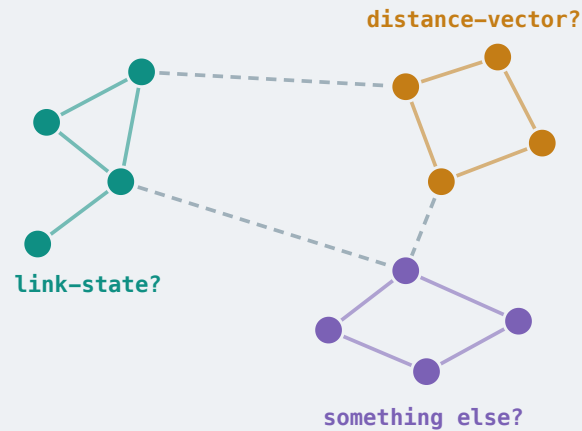
The routers talk to each of their **neighbours** — exactly as before.

This time, instead of learning a global view, each router directly learns, for each destination, **which neighbour is the best next hop to forward to** for that destination.

Eventually each router builds its forwarding table out of that (next lecture).

Who runs the routing protocol?

A network is a great many routers — and they are not all run by the same people.



colour = the organisation that operates the router

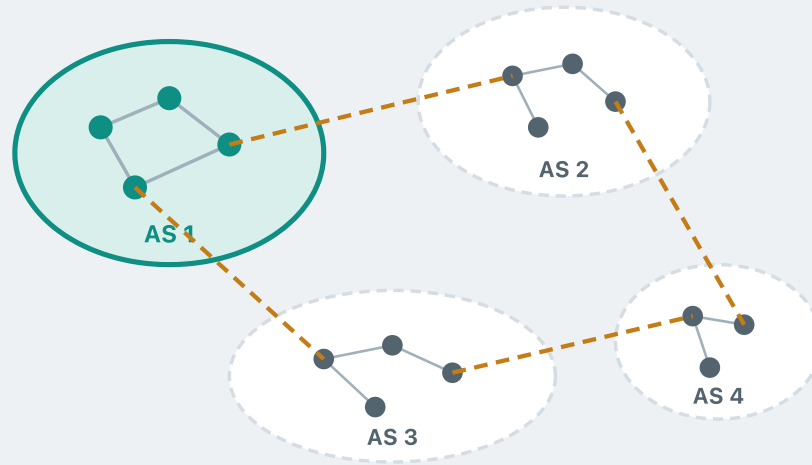
One operator may want **link-state**. Another may prefer **distance-vector**. A third may not want to publish how its network is wired at all.

So **who decides** which routing protocol runs? Nobody is in charge of the Internet, so nobody gets to impose one answer.

The way out is to stop thinking of the Internet as a whole. It is a **network of subnetworks**.

So find the smallest piece that has the **autonomy** to decide for itself — the Internet is a network of **those**.

The unit of the Internet: the AS



An **autonomous system** is a set of routers and networks...

managed by a **single organisation**

running under a **common routing policy** internally

has **full autonomy** over which routing protocol to run within its own network

CONCRETELY

The AS you are inside right now

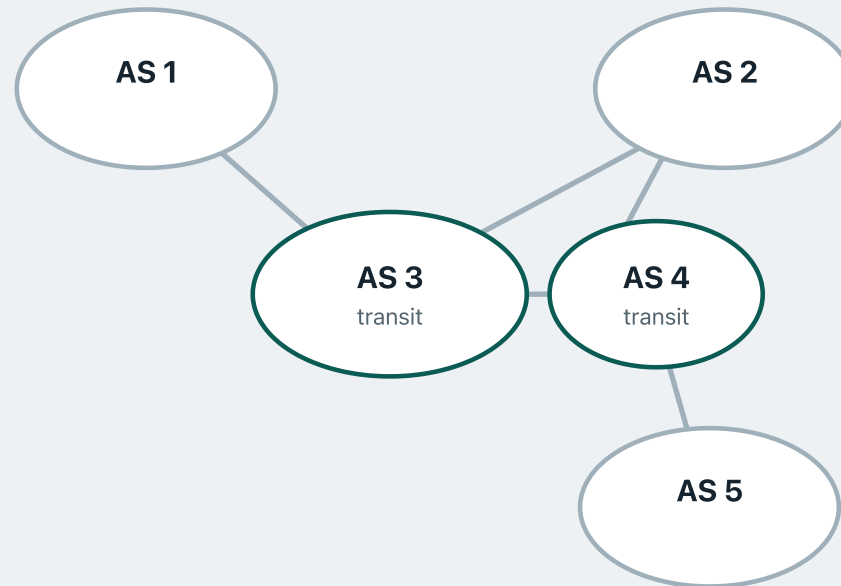
AS handle	AS9432
ASN name	CANTERBURY-AS
organisation	University of Canterbury
registry	APNIC (Asia-Pacific)
IPv4 prefixes	132.181.0.0/16 202.36.178.0/23

asnlookup.com, retrieved July 2026

AS9432 is the **AS number** — the handle every other AS on the Internet uses to refer to this one.

The **/16** is the **subnet mask**: the first 16 bits name the network, so every 132.181.x.x address sits inside it.

Three kinds of AS — by how traffic may flow

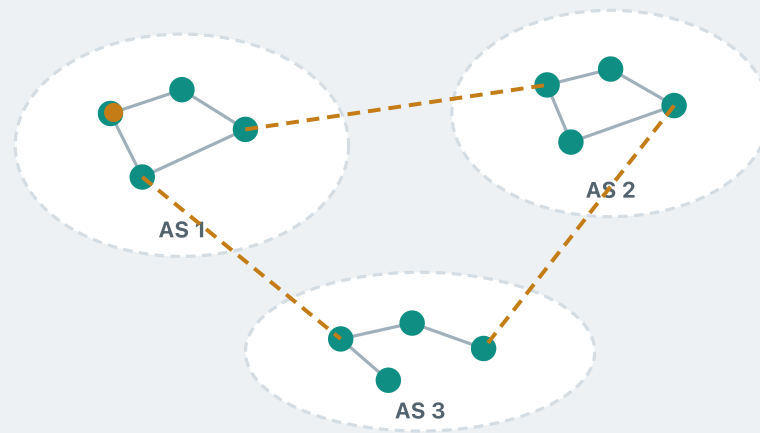


Stub — one connection to one **provider** (an AS that sells connectivity to other ASes).
e.g. AS 3 is the provider to AS 1, which is a stub.

Multi-homed — several providers, but carries **no through-traffic**.
e.g. a datagram may not go from AS 3 through AS 2 to AS 4.

Transit — connects ASes and **carries others' traffic**. That service is the product.
(e.g. One NZ, Verizon)

Intra-AS routing vs Inter-AS routing



intra-AS routing protocol

Routing **within** a single AS — figure out the path a packet travels from one router to another router inside the same AS.

Each AS has **full autonomy** over which protocol to run internally.

inter-AS routing protocol

Routing **between** autonomous systems — figure out the path a packet travels from one AS to another.

Intra-AS and Inter-AS protocols

intra-AS routing protocols

RIP (Routing Information Protocol)

Algorithm family: **distance-vector**

OSPF (Open Shortest Path First)

Algorithm family: **link-state**

IS-IS (Intermediate System to Intermediate System)

Algorithm family: **link-state**

inter-AS routing protocols

BGP (Border Gateway Protocol)

Algorithm family: **path-vector**

We will look at these protocols themselves in Module 2.

RECAP

The routing problem — what we covered

Routing finds a path from every source to every destination.

It builds the **forwarding tables**; **forwarding** just reads one row and sends on.

Two families of algorithm: **link-state** and **distance-vector**.

No one controls the Internet — it is a network of **autonomous systems**.

Each AS has autonomy over its routing, so routing splits: **intra-AS** and **inter-AS**.

MODULE 1

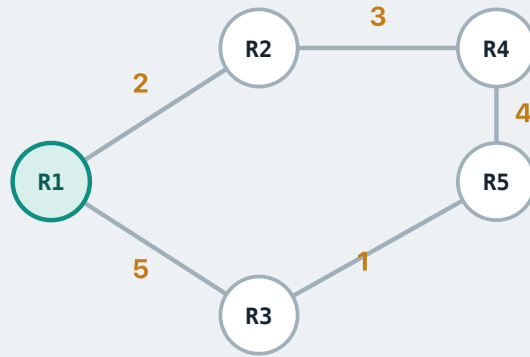
Shortest Paths & Distance-Vector Routing

1 · The Routing Problem

2 · Shortest Paths & Distance-Vector

3 · Dijkstra & Link-State Routing

Networks as graphs



Model the network as a **weighted graph**.

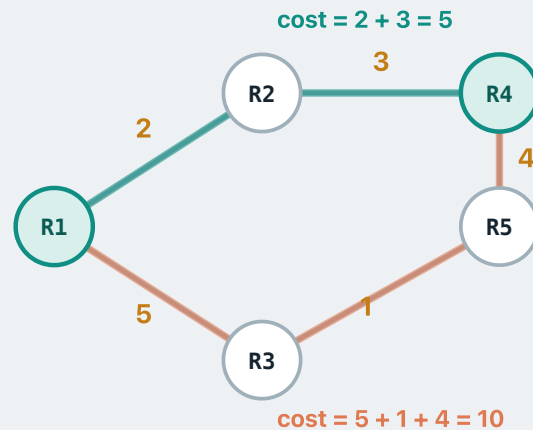
Routers are **vertices**

Links are **edges**

Link metrics are **weights / costs** — can represent inverse bandwidth, delay, or admin-assigned cost. We use these terms interchangeably: *least-cost path* = shortest path by total weight.

MOTIVATION

Why shortest paths?



Suppose R1 wants to send a packet to R4. Two options:

Path A: R1 → R2 → R4 (cost 5)

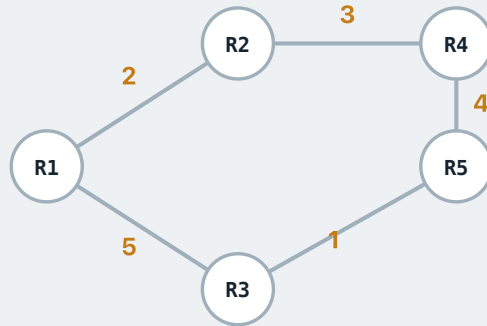
Path B: R1 → R3 → R5 → R4 (cost 10)

Path A wins on every count — lower delay, lower economic cost, better use of bandwidth.

COMMON ROUTING PRINCIPLE

Routing favours the **least-cost path**.

Computing shortest paths



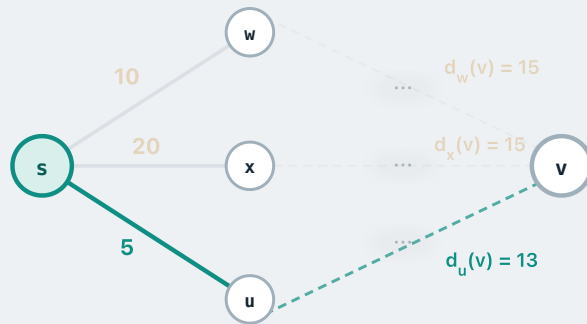
For a small network like this, we can spot the shortest paths by inspection.

But real networks have **thousands to millions** of routers — finding the shortest path for every source–destination pair is no longer obvious.

We need a systematic algorithm — a **shortest-path algorithm**.

Relaxation is the core building block of the common shortest-path algorithms.

The core operation: relaxation



dest	next _s (.)	d _s (.)
x	x	20
v	u	18
...	...	...

Goal: find the shortest path from **s** to **v**.

So far the best known cost to **v** is **35**, with next hop **x**.

Now on to the next neighbour, **w**.

Ask **w** how far it is from **v**. It answers $d_w(v) = 15$.

So the least cost to **v** with **w** as the first hop is $c(s, w) + d_w(v) = 10 + 15 = \mathbf{25}$

Our best known cost to **v** is 35, and going through **w** offers 25 — cheaper.

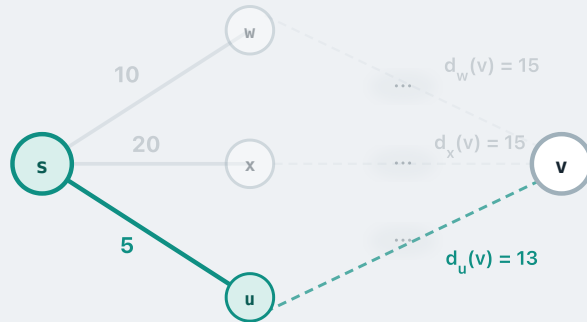
So we take it too: $next_s(v) = w, d_s(v) = 25$. **RELAXATION**

and the other neighbour **u**: $c(s, u) + d_u(v) = 5 + 13 = \mathbf{18}$

$18 < 25$ — better still, so relax again. **RELAXATION**

And update both again: $next_s(v) = u, d_s(v) = 18$.

Relaxation: the formal rule



$d_s(v)$ = best known cost $s \rightarrow v$ · $c(s, w)$ = link cost · $next_s(v)$ = next hop
 $d_s^*(v)$ = true optimal cost $s \rightarrow v$

RELAXATION

Pick any neighbour **w**: if the cost to v through w beats the best known cost, take it.

```
if  $c(s, w) + d_w(v) < d_s(v)$ :  
     $d_s(v) \leftarrow c(s, w) + d_w(v)$   
     $next_s(v) \leftarrow w$ 
```

OBSERVATION

- Write $d_s^*(v)$ for the **optimal** cost from s to v — the cost of a genuinely shortest path.
- Clearly $d_s(v) \geq d_s^*(v)$: the estimate is always **at least** the optimal cost.
- Each relaxation can only ever **decrease** $d_s(v)$.
- So intuitively, keep relaxing for long enough and $d_s(v)$ should end up at $d_s^*(v)$.

Bellman-Ford

BELLMAN-FORD

```

1 Initialisation:
2    $d_s(s) = 0$  for every node  $s$ 
3    $d_s(u) = c(s,u)$  if  $u$  is a neighbour of  $s$ , else  $\infty$ 
4
5 repeat:
6   for each node  $s$ : ← distribute this loop
7     for each neighbour  $w$  of  $s$ :
8       for each destination  $u$ :
9         if  $c(s,w) + d_w(u) < d_s(u)$ : ← relax
10           $d_s(u) = c(s,w) + d_w(u)$ 
11           $next_s(u) = w$ 
12   if no changes: stop

```

So we now have a shortest-path algorithm. How does **distance-vector routing** use it?

Not in this shape — it rests on one **observation** about the inner loop.

IMPORTANT OBSERVATION

To run the inner relaxation (lines 7–11) for itself, node s only needs its **own link costs** $c(s,w)$ and its **neighbours' distance vectors** $d_w(u)$. That is purely **local** information — s never has to learn, for example, the cost of a link **two hops away**.

The distance-vector protocol

```

1  Initialisation (at node s):
2     $d_s(s) = 0$ 
3     $d_s(u) = c(s,u)$  if u is a neighbour, else  $\infty$ 
4     $next_s(u) = u$  if u is a neighbour, else nil
5     $d_w(u) = \infty$  for every neighbour w, every destination u
      ← the last vector received from w, stored at s
6
7  on receiving a vector, or a link cost change:
8    for each destination u:
9      q = nexts(u)
10     if  $c(s,q) + d_q(u) > d_s(u)$ : ← forced accept
11        $d_s(u) = c(s,q) + d_q(u)$ 
12     for each neighbour w of s:
13       if  $c(s,w) + d_w(u) < d_s(u)$ : ← relax
14          $d_s(u) = c(s,w) + d_w(u)$ 
15          $next_s(u) = w$ 
16     if any  $d_s(u)$  changed:
17       send  $d_s(u)$  to all neighbours

```

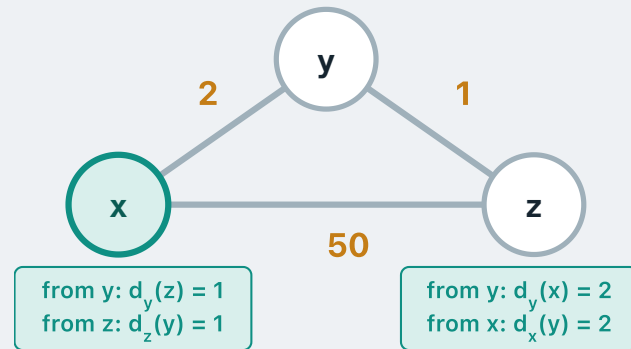
FORCED-ACCEPT RULE

If the update comes from your **current next-hop** for a destination, accept unconditionally — even if the cost is worse.

Why?

Your route to that destination runs **through** that neighbour. If their cost changes, yours changes with it — your old estimate is simply no longer true.

Distance-vector step-by-step



Z RELAXES FOR DESTINATION Y

via x: $c(z,x) + d_x(y) = 50 + 2 = 52$, worse than the **1** it already has — **no change**. y is already direct to both, so it changes nothing either.

x's table

DEST	$D_x(\cdot)$	$NEXT_x(\cdot)$
y	2	y
z	3	y

y's table

DEST	$D_y(\cdot)$	$NEXT_y(\cdot)$
x	2	x
z	1	z

z's table

DEST	$D_z(\cdot)$	$NEXT_z(\cdot)$
x	3	y
y	1	y

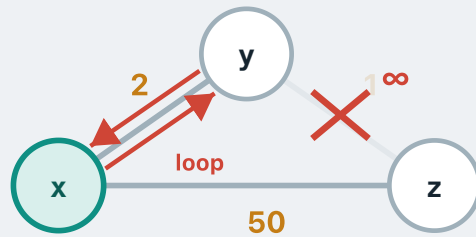
The count-to-infinity problem

x

DEST	$D_x(\cdot)$	$NEXT_x(\cdot)$
y	2	y
z	3	y

y

DEST	$D_y(\cdot)$	$NEXT_y(\cdot)$
x	2	x
z	5	x



Y RELAXES THROUGH ITS NEIGHBOURS

$d_y(z)$ is now ∞ . y runs the relaxation loop.

```

12 for each neighbour w of s:
13   if  $c(s,w) + d_w(u) < d_s(u)$ : ← relax
14      $d_s(u) = c(s,w) + d_w(u)$ 
15      $next_s(u) = w$ 

```

through x: $c(y,x) + d_x(z) = 2 + 3 = 5$

y finds $d_x(z) = 3$ from x's earlier advertisement, stored locally at y.

$5 < \infty \rightarrow d_y(z) = 5, next_y(z) = x$

y has no idea it just created a **loop**. x's route to z runs through y, and y's route to z now runs through x, so the two of them will hand the same packet back and forth. Neither can see it: each only ever sees a cost, never the path behind it.

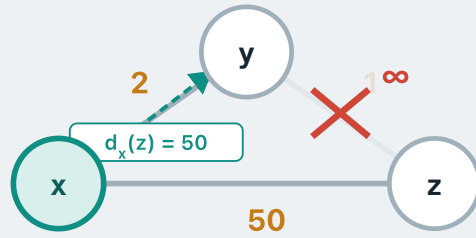
Poison reverse

x

DEST	$D_x(\cdot)$	$NEXT_x(\cdot)$
z	50	z

y

DEST	$D_y(\cdot)$	$NEXT_y(\cdot)$
z	52	x

**No.**

Can you think of a case where it fails?

What we covered

The goal: routing favours the **least-cost path** — which is the shortest-path problem.

Relaxation: the core mechanism in shortest-path algorithms — if a neighbour offers a cheaper cost, take it, and make that neighbour your next hop.

The distance-vector protocol: a **distributed** Bellman–Ford. Every node performs relaxation independently and repeatedly, using **only what its direct neighbours tell it** — and passes its own result back to those same neighbours.

So DV is a purely **distributed, local** algorithm: **no router ever needs the full topology** of the network.

Its limitation: count-to-infinity. Poison reverse is a heuristic that mitigates it, but does not completely solve it.

Dijkstra's Algorithm & Link-State Routing

A faster shortest-path algorithm — and the flooding mechanism that makes it possible.

1 • The Routing Problem

2 • Shortest Paths & Distance-Vector

3 • Dijkstra & Link-State Routing

RECALL

The distance-vector approach

We have seen the **distance-vector protocol** — Bellman-Ford, distributed.

Key property: each router only talks to its **direct neighbours**. No router needs the global topology — each one performs relaxation using its own link costs and the distance vectors it receives.

But it has one limitation: **count-to-infinity**. A distance vector reports **distances, not paths**, so a router cannot see that the route it is offered runs back through itself.

Poison reverse helps, but not for loops of three or more nodes.

Can a different routing approach avoid this problem?

What if every router had the full map?

A simple idea: what if every router knew the **complete network topology**? Then each router could compute its shortest paths to all destinations **directly**.

This avoids DV's limitation: **no count-to-infinity** — routers share the **map**.

Each router then works out the **shortest path** over that whole topology itself — and a shortest path can never contain a loop, since going round in a circle only adds cost.

This is the idea behind **link-state routing**.

How link-state routing works

Step 1 • discover

Each router discovers its **neighbours** and knows the **cost of each of its own links**.

Step 2 • flood

Each router broadcasts its local link information — a **link-state advertisement (LSA)** — to **every other router**. This process is called **flooding**.

Step 3 • compute

Every router assembles all LSAs into the **same complete map** and runs a **shortest-path algorithm** to build its forwarding table.

Step 4 • re-flood

When a link cost changes or a link fails, the affected router floods a **new LSA**. Every router updates its map and reruns the algorithm.

Next: Step 3 — the shortest-path algorithm. **Later:** Steps 1, 2 & 4.

What algorithm do we need?

Goal: given the full topology, every source node **s** needs the shortest-path cost to every other node **v**, and the **next hop** to reach it.

Bellman–Ford would do the job perfectly well — we have already seen it work, and it does not even need the full topology: the routers only ever talk to their neighbours.

But now every router *does* know the **full topology** — and in that setting there is a better-suited choice: **Dijkstra's algorithm**.

How it works

```

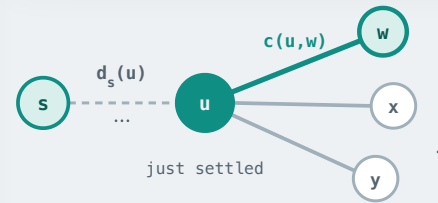
initialise  $d_s(s) = 0$ 
 $d_s(v) = c(s,v)$  if  $v$  is a neighbour of  $s$ , else  $\infty$ 
 $\text{pred}_s(v) = s$  if  $v$  is a neighbour of  $s$ , else nil
 $S = \{s\}$  (settled set)

repeat:
     $u =$  unsettled node with smallest  $d_s(u)$ 
    add  $u$  to  $S$ 
    for each neighbour  $w$  of  $u$ :
        if  $d_s(u) + c(u,w) < d_s(w)$ :  $\leftarrow$  relaxation!
             $d_s(w) = d_s(u) + c(u,w)$ 
             $\text{pred}_s(w) = u$ 
until all nodes settled
  
```

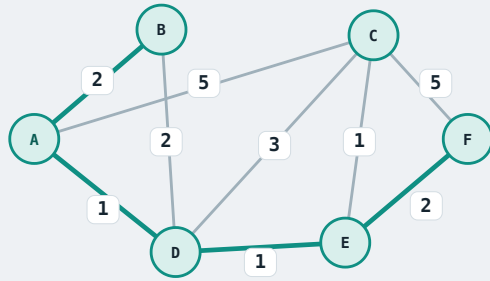
RELAXATION

If it is, take it: $d_s(w)$ becomes $d_s(u) + c(u,w)$, and since we reached w from u , $\text{pred}_s(w) = u$.

That is exactly **relaxation**.



Step-by-step walkthrough



Step 5 of 5

Step	S	$d_s(A)$ $\text{pred}_s(A)$	$d_s(B)$ $\text{pred}_s(B)$	$d_s(C)$ $\text{pred}_s(C)$	$d_s(D)$ $\text{pred}_s(D)$	$d_s(E)$ $\text{pred}_s(E)$	$d_s(F)$ $\text{pred}_s(F)$
init	{A}	0, —	2, A	5, A	1, A	∞	∞
1	{A, D}	—	2, A	4, D	—	2, D	∞
2	{A, D, E}	—	2, A	3, E	—	—	4, E
3	{A, D, E, B}	—	—	3, E	—	—	4, E
4	{A, D, E, B, C}	—	—	—	—	—	4, E
5	{A, D, E, B, C, F}	—	—	—	—	—	—

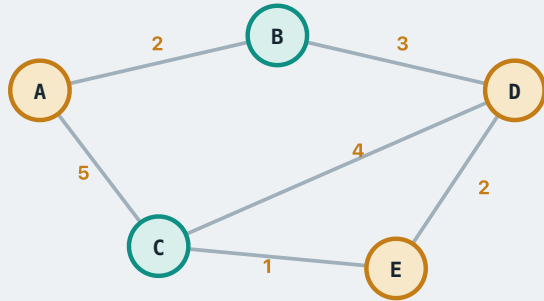
How every router builds its map

Each router starts with only what it knows locally: its **neighbours** and the cost of **its own links**. It packages this as a **Link-State Advertisement (LSA)**.

Each router **floods** its LSA — it sends the LSA to all its neighbours, who forward it to their neighbours, and so on until every other router has received it.

Each router stores all incoming LSAs in its **Link-State Database (LSDB)**. Once all LSAs have propagated, every router's LSDB is **identical** — they all share the same complete topology.

Building the global picture



Step 1

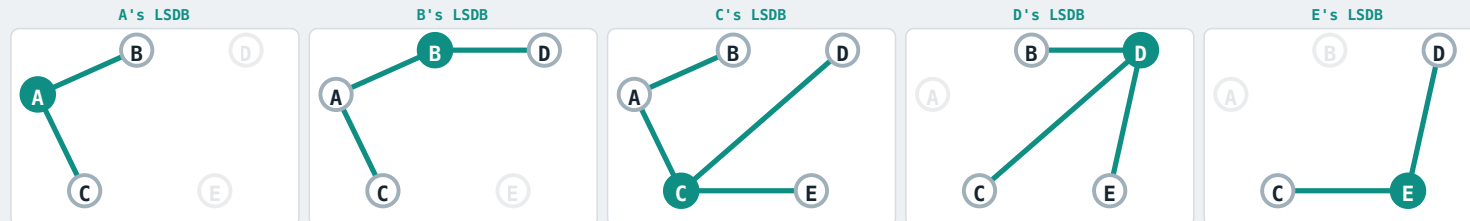
Each router discovers its local neighbours and link costs. Below: the initial **Link-State Database (LSDB)** for every router.

Step 2

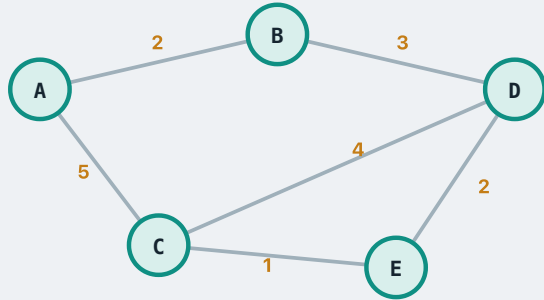
A packages its local view into **LSA_A** and sends it to its neighbours B and C.

Step 3

B and C receive **LSA_A** and update their LSDB — their view now includes A's links.



Building the global picture

**Step 4**

B and C forward **LSA_A** onward: B → D, and C → D and E.

Step 5

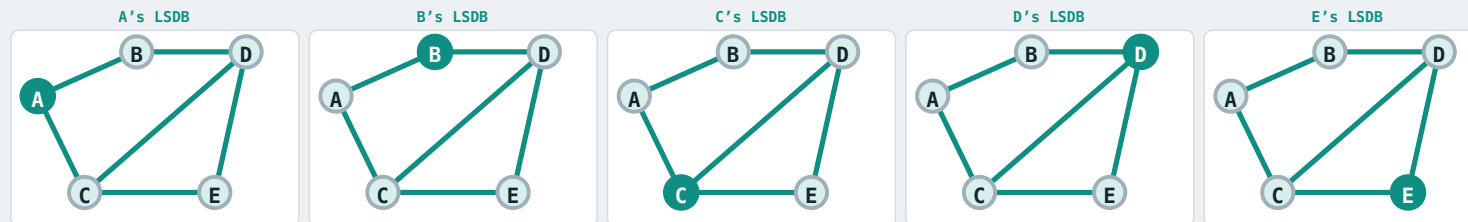
D and E receive **LSA_A** and update their LSDB.

Step 6

This all happens in parallel: B, C, D, E simultaneously advertise their own LSAs.

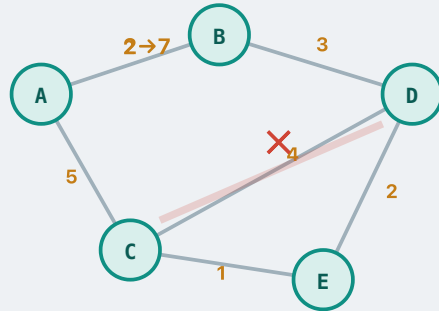
Step 7

After all LSAs propagate, every router's LSDB is **identical** — they all share the same complete topology.



WHEN THE MAP CHANGES

Re-flooding



Why do we need re-flooding?

Topology change — a link goes down, a router crashes, or a new link appears

Cost change — an administrator changes a link weight

Periodic refresh — every ~30 min, each router re-announces its links even if nothing has changed

What happens — the affected router generates a **fresh LSA** and re-floods it. Every router updates its LSDB.

Distance-vector vs Link-state

	DISTANCE-VECTOR	LINK-STATE
Approach	Distributed – only ever talks to its neighbours	Centralised – needs the full topology (LSDB)
Routing loops	Prone (count-to-infinity)	Loop-free (consistent topology)
Memory	Small (only distance vectors)	Large (full topology database / LSDB)

Lineage & colophon

These lecture notes modernise COSC264 material developed by **Dr Barry Wu** (2024–25), building on notes by **Dr Mengmeng Ge** (2023) and **Assoc Prof Dan Kim** (2017), University of Canterbury, and lecture notes by **Prof Aleksandar Kuzmanovic** (CS340, Northwestern University, 2005).